

Relazione su Merge Sort e Quick Sort

Matricola: 952435

Studente: Liberatore Mattia

Scelte Implementative

Merge Sort

L'implementazione del Merge Sort segue l'approccio classico diviso in due funzioni principali:

1. **merge_sort()** - funzione ricorsiva che divide l'array in sottoarray
2. **merge()** - funzione iterativa che fonde due array ordinati

Quick Sort

Per il Quick Sort ho implementato la versione ricorsiva con le seguenti ottimizzazioni:

- **Scelta del pivot mediano:** invece dell'ultimo elemento, utilizzo la strategia "median-of-three" che seleziona il pivot come mediano tra primo, medio e ultimo elemento dell'array
- **Partizione:** implementazione standard con partizionamento in due sezioni

Prestazioni

Ordinamento di Interi

Dimensione Array	Merge Sort (sec)	Quick Sort (sec)	Rapporto
1.000.000	0,25	0,35	1,4x
5.000.000	1,60	2,00	1,25x
10.000.000	3,50	4,00	1,14x
20.000.000	7,00	8,00	1,14x

Ordinamento di Double

Dimensione Array	Merge Sort (sec)	Quick Sort (sec)	Rapporto
1.000.000	0,30	0,36	1,2x
5.000.000	1,80	2,00	1,11x
10.000.000	4,00	4,20	1,05x
20.000.000	8,00	11,50	1,44x

Ordinamento di Stringhe - Analisi Prestazionale

Dimensione Array	Merge Sort (sec)	Quick Sort (sec)	Rapporto
1.000.000	0,50	5,80	11,6x
5.000.000	3,00	281	93,7x
10.000.000	5,80	1582 (fail)	272,8x
20.000.000	14,00	>600 (fail)	>42,9x

Analisi e Considerazioni

Performance su Interi e Double

I risultati dimostrano che per dati numerici (interi e double) i due algoritmi hanno performance comparabili.

Criticità nell'Ordinamento di Stringhe

L'implementazione del Quick Sort mostra **prestazioni drammaticamente inferiori** con il dataset di stringhe, rivelando una **degradazione esponenziale**:

- **1M → 5M**: 5,8s → 281s (aumento di 48,4x per 5x dati - $O(n^2)$)
- **5M → 10M**: 281s → 1582s (aumento di 5,6x per 2x dati - $O(n^2)$)
- **Complessità**: Chiara degradazione a complessità quadratica confermata

Questo comportamento critico è direttamente riconducibile a:

1. **Altissima densità di duplicati**: Il dataset contiene un'elevatissima percentuale di stringhe identiche
2. **Partizionamento binario inefficace**: concentra tutti i duplicati in una sola partizione creando partizioni di dimensioni estremamente disproporzionate

Analisi Comparativa: Partizione Normale vs Mediana di 3

Il confronto tra le due versioni del Quick Sort ha rivelato risultati controintuitivi:

- **Partizione normale**: 5,8s (1M), 281s (5M), 1582s (10M)
- **Mediana di 3**: 6,0s (1M), 324s (5M), 1680s (10M)

La versione con pivot finale ha dimostrato leggere ma consistenti vantaggi prestazionali, attribuibili a:

- **Minore overhead computazionale**: Assenza delle operazioni di selezione della mediana

- **Swap più efficienti:** Minore numero di operazioni di scambio
- **Inefficacia della mediana con duplicati:** La scelta del pivot diventa irrilevante in presenza di altissime percentuali di duplicati

Conclusioni Definite

L'analisi conferma in modo inequivocabile che:

1. **MergeSort dimostra superiorità assoluta**
2. **QuickSort binario fallisce con duplicati:** inutilizzabile per dati reali con ripetizioni (1582s per 10M stringhe). Il problema fondamentale risiede nel meccanismo di partizionamento binario e non nella scelta del pivot.
3. Questo progetto dimostra l'importanza cruciale di: comprendere la natura dei dati, selezionare l'algoritmo appropriato al dominio applicativo, e validare le scelte implementative con test approfonditi su dataset rappresentativi prima del deployment in ambienti produttivi.